Network Resilience Lab

Team 24

Sreerag Aneesh Nair · 24BCE0081

Rini Vaishnavi R S · 24BAI0261

Vellore Institute of Technology | BCSE308P | L35+L36

The problem

- Temporary disconnections can discard nearly completed file uploads.
- Restarting from zero repeats data already transmitted.
- We compare completion, recovery time, redundant bytes and integrity under identical network schedules.

Objectives

- Implement Basic, Retry and Resumable HTTP uploads over TCP.
- Apply repeatable bandwidth limits, request delays and scheduled outages.
- Verify success with SHA-256 and measure recovery without subjective scores.
- Provide a dashboard, event logs and CSV/JSON exports.

What makes this project useful?

- One inspectable testbed combines a disruption proxy, server-confirmed progress and serial trials.
- Each strategy receives a fresh origin, independent clock and equivalent scenario.
- The novelty is the integrated reproducible comparison—not a new upload algorithm.

System architecture

Upload client

Python client sends actual HTTP body bytes using the selected strategy.

Disruption proxy

Shared-rate pacing, per-request delay and connection outage scheduling.

Upload origin

Atomically commits complete requests; returns offsets and SHA-256 digest.

Three upload strategies

BASIC

One whole-file request.

Connection failure ends the trial.

RETRY

Restarts the entire file after failure with exponential backoff.

RESUMABLE

32 KiB chunks. Queries committed offset and resumes from acknowledged progress.

Experimental design

27 uploads

3 scenarios × 3 strategies × 3 trials

0.5 MiB payload

524,288 deterministic bytes; 60-second deadline

Fixed controls

20 ms request delay; up to 12 retries; 32 KiB chunks

Network scenarios

STABLE

2,048 kbps throughout

SLOW

256 kbps throughout

OUTAGE

2,048 kbps; offline from 2 to 7 seconds; then restored

Evaluation metrics

- Verified completion: matching SHA-256 digests at client and origin.
- Redundant bytes = max(0, proxy ingress − committed unique bytes).
- Useful throughput = verified payload bits / elapsed time.
- Recovery time: from network restoration to next increase in confirmed progress.

Stable and slow: completion times

STABLE

Basic: 2.12 s

Retry: 2.12 s

Resumable: 2.61 s

SLOW

Basic: 16.46 s

Retry: 16.46 s

Resumable: 16.95 s

OBSERVATION

All 18 transfers verified. Chunk acknowledgements add overhead without outages.

Five-second outage: results

BASIC

0 / 3 verified

Connection interrupted

RETRY

3 / 3 verified

9.95 s mean

498,347 extra bytes

RESUMABLE

3 / 3 verified

8.50 s mean

4,096 extra bytes

Key finding

99.2%

fewer redundant application bytes with Resumable versus Retry under the tested outage.

Recovery behavior

RETRY

2.95 s mean recovery time

95.05% application overhead

RESUMABLE

1.01 s mean recovery time

0.78% application overhead

IMPORTANT

Retry acknowledges whole-file completion; Resumable acknowledges chunks. Recovery metrics differ in granularity.

Validation and limitations

- 18 integration tests passed; all 27 evaluation records and exported digests checked.
- Localhost software proxy—not packet-loss or real Wi-Fi emulation.
- Only one payload size, chunk size and fixed execution order; three trials per group.
- In-memory progress is not preserved after an origin restart.

Conclusion and future scope

- Basic failed under the tested interruption; Retry and Resumable both succeeded.
- Resumable saved retransmitted application data, but had extra overhead on stable links.
- Future work: two-computer tests, varied chunk sizes, randomized trials and persistent server state.

Thank you